

Module 9: Classes and Objects

Sarah Foss

Objectives

Explain what a class is, its role in defining properties and methods, and how it serves as a blueprint for creating objects. Describe how object references differ from objects themselves and demonstrate how to instantiate and interact with objects.

Identify and distinguish between instance variables/methods and static variables/methods in a class. Explain how instance members are associated with individual objects while static members are shared across all instances.

Describe the purpose and usage of visibility modifiers to control access to class members. Demonstrate how to create getter and setter methods to provide controlled access to private data, and explain the concept of immutable objects and their benefits for data integrity.

Illustrate the use of Java wrapper classes for primitive data types, and describe the concepts of auto-boxing and unboxing. Differentiate between mutable and immutable string classes (String and StringBuilder) and explain when to use each.

Objects

In Java, an **object** represents something real-world—like a person, place, or thing—that has details (attributes) and it can perform actions (behaviours).

When you make an object, Java sets aside memory for its data and gives you access to the methods that let you interact with it.

Object-oriented programming (OOP) is all about building programs using objects that represent real-world things.

For example, a student, a desk, a circle, a button, or even a loan can be thought of as objects.

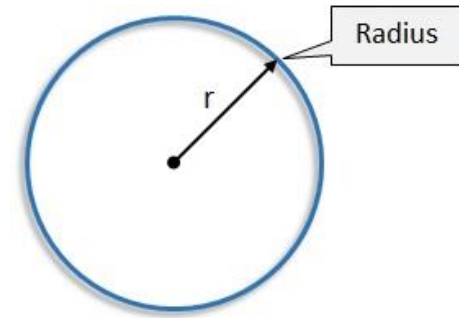
Object
[actual object]



Location, State, and Behaviours

Each object has:

- **A unique identity:** Every object is stored in a specific memory location, which makes it unique.
- **A state:** This is made up of data fields (also called attributes or properties) that hold the current values for its details, like a car's color, a student's name, or the radius of a circle.
- **Behaviours:** These are the actions an object can do, defined by methods, like a car driving, a student studying, or a button being clicked.



For example, you can define the state of a circle by giving it a property like `radius`, and you can define its behaviors by creating methods such as `getArea()`, `getPerimeter()`, and `setRadius(radius)` for circle objects.

Classes

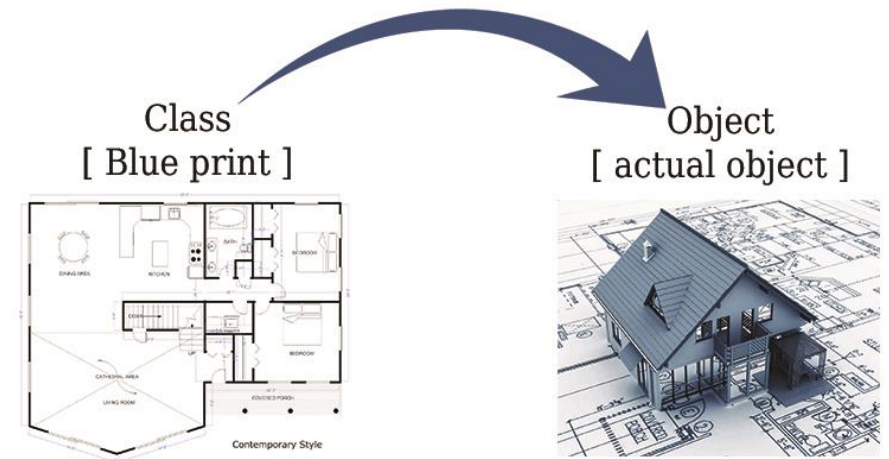
Classes are like templates that describe what similar objects will look like and what they can do.

Think of a class as a blueprint that outlines the data and actions for objects of that type.

An object is one specific **instance** of a class, created based on this blueprint.

You can make as many objects as you need from a single class.

When you create an object, it's called **instantiation**.



Constructor

A **constructor** is a special type of method in a class that's called when you create a new object.

Its purpose is to initialize the object, setting up its initial values and any necessary setup steps.

Syntax:

```
class ClassName {  
    ClassName(parameter list) {  
        //constructor body  
    }  
}
```

Constructor Rules

- A constructor must have the same name as the class.
- It has no return type, not even `void`.
- A constructor with no parameters is referred to as a ***no-arg constructor***.
- A constructor is invoked when an object is created using the `new` keyword.
- You can define multiple constructors with different parameters (constructor overloading).
- If no constructor is defined, Java provides a default no-argument constructor.

Invoking a Constructor

To create an instance of a class, invoke the constructor using the new keyword

Syntax:

```
new ClassName (arguments) ;
```

Examples:

```
new Circle() ;    //invokes the no-arg constructor
```

```
new Circle(3.0) ;
```

Default Constructor

If a class is defined without a constructor, Java automatically provides a **default constructor**.

A default constructor is a no-args constructor with an empty body.

A default constructor is only provided if there is no constructor explicitly defined in the class.

Classes are Datatypes

When you create a class, you're essentially defining a new **datatype** that can hold specific kinds of data (attributes) and perform actions (methods).

For instance, if you define a Car class, you're creating a new type called Car that can store car-related data (like color and model) and perform car-related actions (like `start()` or `drive()`).

With classes, you can create objects that represent instances of this new type, giving you the flexibility to model almost anything you can think of in your program.

Reference Variables

Just like with arrays, when you instantiate an object, space is allocated for it in the heap, which remains there until the object is no longer needed. Once no references to the object exist, Java's garbage collector will automatically remove it to free up memory.

Objects are accessed via the object's reference variables.

The dot operator (.) in Java is used to access members (attributes and methods) of an object or class. It connects the object or class name to the specific variable or method you want to use.

Accessing Object Attributes:

You can use the dot operator to access or modify an object's attributes.

```
myCar.color = "blue";
```

Calling Object Methods:

You can use the dot operator to call methods of an object

```
myCar.start();
```

UML Class Diagrams

UML (Unified Modeling Language) **class diagrams** are a powerful tool for designing and visualizing the structure of a Java application before actually writing code.

They help developers plan how classes will interact, defining relationships and structures in an organized way.

In the class diagram, the attribute is denoted as

`dataFieldName: dataFieldType`

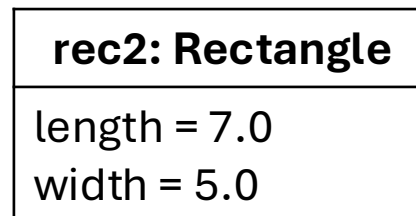
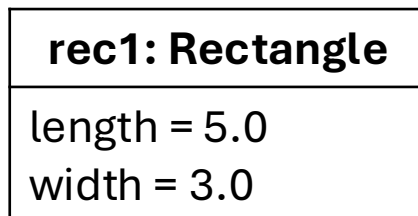
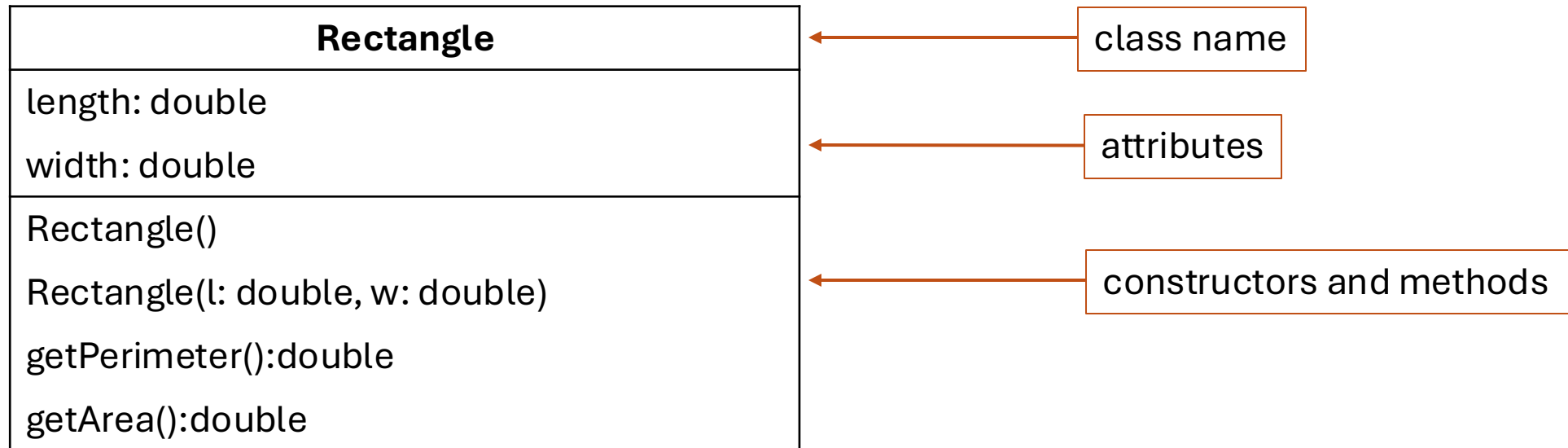
The constructor is denoted as

`ClassName (parameterName: parameterType)`

The method is denoted as

`methodName (parameterName: parameterType): returnType`

UML Class Diagrams



objects

Reference Type Data Fields

Data fields can be of reference types.

```
public class Book {  
    String title;           // Reference to a String object  
    Author author;         // Reference to an Author object  
    String[] genres;       // Reference to an array of strings  
}
```

Remember: A reference type is a variable that doesn't store the actual value of an object but rather a reference (or memory address) that points to the location of the object in memory.

null

`null` is a special literal that represents the absence of a value for reference types

When a variable is set to `null`, it means it doesn't point to any object or data in memory.

A runtime occurs when you attempt to use a null reference as though it were an actual object. This `NullPointerException` can occur in situations like:

- Calling a method on a null object
- Accessing or modifying a field of a null object

To avoid this error, make sure to explicitly initialize the variable or use `if` statements to check if a variable is `null` before using it.

Note: An empty String (`""`) is not null; it's simply a string with zero characters.

Data Field Default Values

Unlike local variables, which need to be explicitly initialized, data fields have default values.

The default value of an attribute:

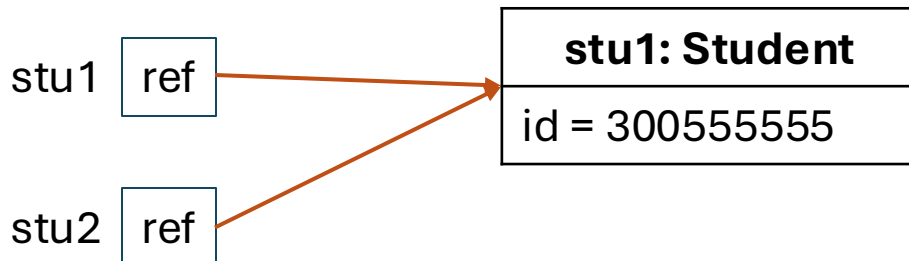
- `0` for a numeric type
- `false` for a boolean type
- `'\u0000'` for a char type
- `null` for a reference type

Copying Reference Variables

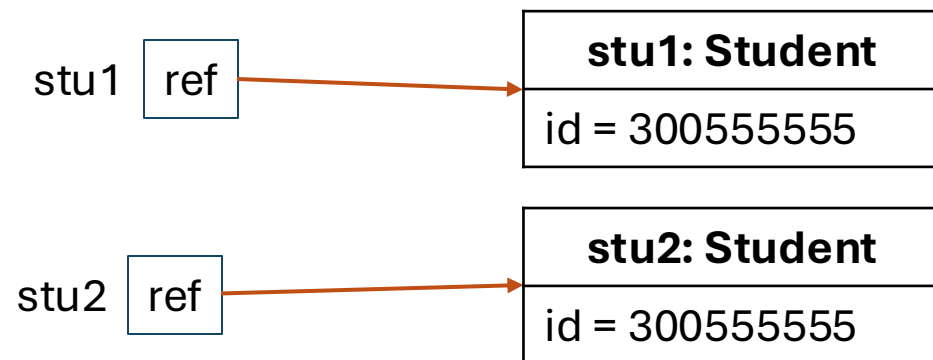
When a variable is a reference type (like a custom class or an array), assigning it directly would only copy the reference, meaning both objects would point to the same object in memory.

To create a true copy, create a new instance of each referenced object and set the attributes accordingly.

```
Student stu1 = new Student(300555555);  
Student stu2 = stu1;
```



```
Student stu1 = new Student(300555555);  
Student stu2 = new Student(stu1.id);
```



Instance Variables

Instance variables are variables that belong to specific object (instance of a class).

Each object created from a class has its own copy of instance variables, which hold the state or data specific to that object.

```
public class Circle {  
    double radius;    //instance variable  
    ...  
}
```

Instance Methods

Instance methods are methods that belong to an instance of a class.

Instance methods require creation of an instance of the class to be called.

They are used to operate on the instance variables of the class, allowing each object to have its own behavior and interact with its own state.

```
Rectangle myCircle = new Circle(3);  
double area = myCircle.getArea();
```

Static Variables

Static variables (also known as class variables) are variables that belong to the class itself, rather than to any individual instance of the class.

- They are shared across all instances of the class, meaning there is only one copy of a static variable, regardless of how many objects are created from the class.
- They are typically used for constants or for data that should be the same across all instances (e.g., a counter, configuration settings).
- They can be accessed directly using the class name.
- Static variables are initialized when the class is loaded and persist as long as the class is loaded.
- To declare static variables, constants, and methods, use the `static` modifier.

Static Methods

Static methods are methods that belong to the class itself rather than to any instance of the class.

This means they can be called without creating an object of the class.

Static methods are useful for functionality that is not dependent on the specific state of any object.

Static methods cannot access non-static (instance) variables or methods directly.

```
Circle.getNumberOfCircles()
```

```
public class Circle{
    double radius; //instance variable
    static int numberOfObjects = 0; // static variable
    final static double PI = 3.14159265358979323846; // static constant

    Circle(){ // no args constructor
        radius = 1;
        numberOfObjects++;
    }
    Circle(double newRadius){ // constructor
        radius = newRadius;
        numberOfObjects++;
    }
    static int getNumberOfObjects(){ // static method
        return numberOfObjects;
    }
    double getArea(){ // instance method
        return radius * radius * Circle.PI;
    }
    double getPerimeter(){ //instance method
        return 2 * radius * Circle.PI;
    }
    public void setRadius(double newRadius){ //instance method
        if(newRadius >= 0)
            radius = newRadius;
    }
    public double getRadius(){ //instance method
        return radius;
    }
}
```

Accessing Static Variables and Methods

Static variables and methods belong to the class itself rather than to instances of the class. This means you can access them using the class name rather than an object instance.

```
Circle.PI
```

```
Circle.getNumberOfObjects()
```

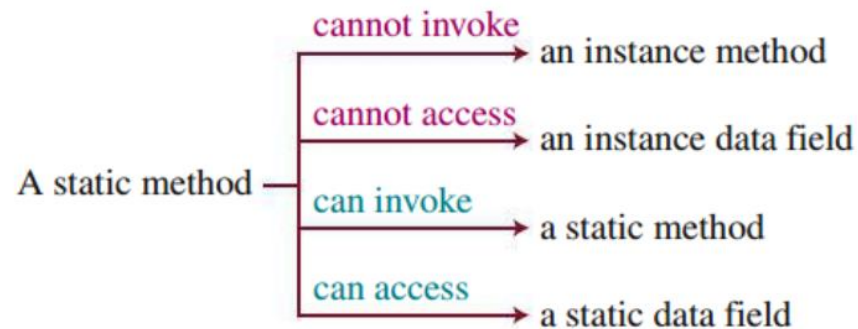
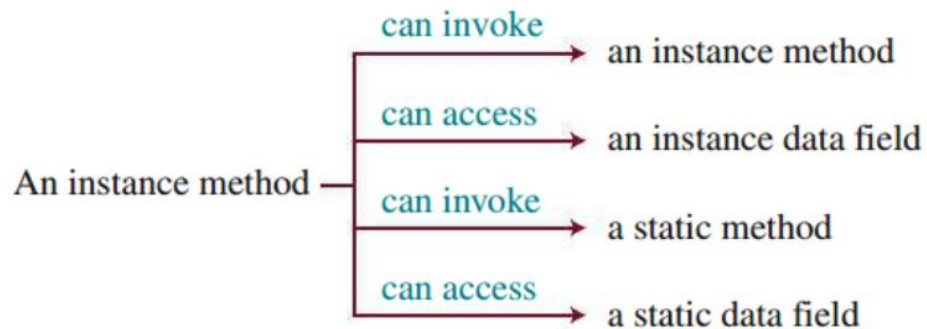
Why Use Class Name for Access?

- **Readability and Intent:** Using the class name clearly shows that the member belongs to the class and isn't tied to any particular instance.
- **Avoiding Instance Requirements:** Since static members aren't tied to an object, you don't need to create an instance to access them, which can be more efficient.

Instance and Static Availability

An instance method can access both instance and static methods and data. This is because instance methods have a link to a specific object/

A static method can access only other static methods and static data fields. A static method cannot access instance methods or instance fields directly, because it doesn't belong to any specific object.



Instance vs Static?

Deciding whether to declare a member (variable or method) as static or instance depends on whether it belongs to the class as a whole (shared by all instances) or to each individual instance (unique to each object).

Declare a **static member** when:

The value or functionality is shared among all instances of the class.

- *For example, a counter that tracks the total number of objects created or a constant value that remains the same across all objects, like π in a Math class.*

The data or method does not rely on any specific instance.

- *For example, utility methods like mathematical functions (e.g., `Math.sqrt()`) or general-purpose methods that don't need object-specific data.*

You want to define constants.

- *Constants, like `final static double PI = 3.14159;`, should be static because they are shared, fixed values relevant to the class as a whole.*

Instance vs Static?

Declare an **instance member** when:

Each object needs its own unique copy of a variable or method functionality.

- *For example, in a Person class, name and age should be instance variables because each person has their own name and age.*

The data or method depends on specific object properties.

- *For instance, an instance method that prints a person's details (printInfo()) should be instance-based because it's displaying information unique to that person.*

Scope and Lifetimes of Variables

Local, static, and instance variables each have different scopes and lifetimes.

Local variables:

Scope: They are accessible only within the method, constructor, or block where they are defined.

Lifetime: They exist only while the method or block is executing. Once the method finishes, the local variable is destroyed, and its memory is freed.

Instance variables:

Scope: They are accessible by all non-static methods in the class. When you create an object of the class, each object has its own copy of the instance variables.

Lifetime: They exist as long as the object exists. When the object is destroyed, the instance variables are also destroyed.

Static variables:

Scope: They are accessible by all methods within the class, whether static or instance methods. Outside the class, they can be accessed by using the class name

Lifetime: Static variables are created when the class is loaded and destroyed when the program ends. They remain in memory for the duration of the program and are shared across all instances of the class.

Packages

A package in Java is a collection of related classes and other related files.

Packages provide a logical grouping for code, similar to folders in a file system.

Types of Packages:

Built-in Packages: Java comes with built-in packages in the standard Java library, like `java.util`, `java.io`, and `java.lang`. These packages contain core classes and interfaces needed for general programming, such as data structures, file handling, and mathematical operations.

User-defined Packages: You can create custom packages to organize your own classes based on functionality, such as `com.myapp.models` or `com.myapp.util`

Importing Packages

If you want to use classes from a different package, you need to import them.

Single Class Import: Import a specific class from a package.

```
import java.util.Scanner;
```

Wildcard Import: Import all classes from a package (not recommended if you only need a few classes).

```
import java.util.*;
```

Static Import: Import static members (variables or methods) directly, allowing you to use them without the class name.

```
import static java.lang.Math.PI;
```

User-Defined Packages

To create a package, add the package statement as the first line in your file, followed by the package name:

```
package packageName;
```

If a class is defined without the package statement, it is said to be placed in the **default package**.

Visibility Modifiers

Visibility modifiers (or access modifiers) determine which classes, methods, and variables are accessible in different parts of a program.

They help control the level of access other classes have to certain members, enhancing data protection.

No keyword (default access)

The class, data, or method is visible to any class in the same package.

public

The class, data, or method is visible to any class in any package.

private

The data or methods can be accessed only by the declaring class.

Visibility Modifiers

Modifier	Accessible Within Same Class	Accessible Within Same Package	Accessible from Anywhere
public	Yes	Yes	Yes
default	Yes	Yes	No
private	Yes	No	No

Encapsulation

Encapsulation is one of the core principles of object-oriented programming (OOP).

Encapsulation hides an object's internal state from the outside world, exposing only what is necessary.

This is achieved by marking variables as **private** and providing controlled access through public methods, commonly known as **getters** and **setters**.

Data hiding helps protect the data from unintended modifications, ensuring that changes to an object's state go through well-defined methods.

Getters and Setters (Accessors and Mutators)

A private data field cannot be accessed by an object from outside the class that defines the private field. However, we often need to get or modify a data field.

To make a private data field accessible, provide a getter (accessor) method to return its value.

To enable a private data field to be updated, provide a setter (mutator) method to set a new value.

A getter (accessor) method has the following signature:

```
public returnType getPropertyName()  
public boolean isPropertyName() //special name for boolean getter
```

Example:

```
public double getRadius()
```

A setter (mutator) method has the following signature:

```
public void setPropertyName(dataType propertyValue)
```

Example:

```
public void setRadius(double radius)
```

this Keyword

In Java, the `this` keyword is a special reference that points to the current object—the instance of the class where the code is currently running.

It's very useful in a variety of situations, especially when you need to distinguish between instance variables and method parameters or access the current object's methods from within the class itself.

this Keyword

Distinguishing Between Instance Variables and Parameters

If a method's parameters have the same names as instance variables, this helps differentiate them.

```
public class Person {  
    private String name;  
    public Person(String name) {  
        //name = name;  // would not change the value of the instance variable  
        this.name = name; // 'this.name' refers to the instance variable  
    }  
    public void setName(String name){  
        this.name = name;  
    }  
    public String getName(){  
        return this.name;  
    }  
}
```

this Keyword

Calling Other Constructors

The `this` keyword can also be used to call another constructor within the same class.

This is called **constructor chaining** and is useful when you have multiple constructors with different parameters.

```
public class Person {  
    private String name;  
    private int age;  
    public Person(){  
        this(""); // Calls the constructor with one parameter  
    }  
    public Person(String name) {  
        this(name, 0); // Calls the constructor with two parameters  
    }  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

toString() Method

The `toString()` method is used to create a string representation of an object.

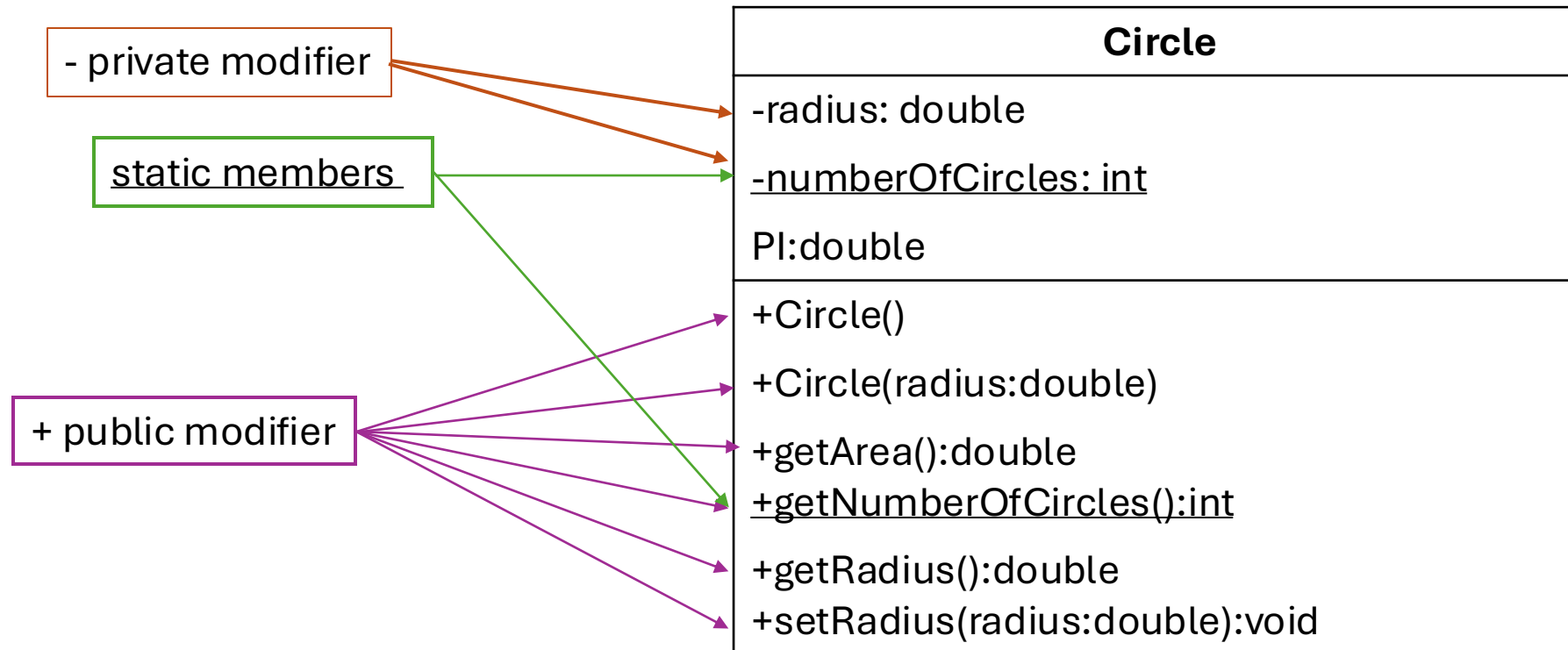
By default, when you print an object or concatenate it with a string, Java calls the object's `toString()` method.

However, the default `toString()` implementation usually just returns a string like `ClassName@45A3E`, which isn't very informative.

To make the output more useful, you can define your own `toString()` method in your class to return meaningful information about the object.

```
public String toString() {  
    return name + ", " + age;  
}
```

UML Class Diagrams



Passing References To Methods

Just like with arrays, when you pass an object to a method, you're actually passing a copy of the reference to that object rather than the object itself.

Effect of Passing References:

- If you pass an object reference to a method and modify the object (e.g., changing data fields of the object), those changes persist after the method completes, as both the caller and the method refer to the same object.
- However, reassigning the reference inside the method will not affect the original reference outside the method. If you assign a new object to the reference, it only affects the local reference inside the method.

Passing References To Methods

```
public class ReferenceExample {
    public static void main(String[] args) {
        Car myCar= new Car("Mazda", "red");
        paintCar(myCar);
        System.out.print(myCar);
        changeCar(myCar);
        System.out.print(myCar);
    }
    static void paintCar(Car car) {
        car.setColor("green");
    }
    static void changeCar(Car car) {
        car = new Car("Jeep", "black"); // Only changes the local reference within the method
    }
}
```

Arrays of Objects

Arrays can hold more than primitive datatype, they can hold reference types as well.

When you create an array of reference types, all of the elements will be `null` by default.

You must explicitly assign each element of the array with an instance of that class.

```
Car[] cars = new Car[3]; // all elements will be null
```

```
cars[0] = new Car("Mazda", "red");  
cars[1] = new Car("Honda", "blue");  
cars[2] = new Car("Ford", "green");
```

```
for (Car car : cars) {  
    System.out.println(car);  
}
```

Immutable Classes and Objects

An immutable class is a class whose instances (objects) cannot be modified after they are created.

Once you create an immutable object, you cannot change its data fields or its state.

This concept is especially useful for maintaining data consistency and avoiding unexpected changes in objects.

Note: String is an immutable class.

Immutable Classes and Objects

To create an immutable class:

1. All data fields must be private
2. No setter methods
3. No getter methods for mutable data fields

Immutable Classes and Objects

```
public class Car {  
    private String name;    //immutable data field  
    private String color;    //immutable data field  
    private ManufactureDate date; //mutable data field  
    public Car(String name, String color) {  
        this.name = name;  
        this.color = color;  
        this.date = new ManufactureDate(10, 10, 2024);  
    }  
    public String getName() {    return this.name;    }  
    public String getColor() {    return this.color;    }  
    //public ManufactureDate() { return this.date; }    // must not return mutable field  
}
```

Javadoc

Javadoc is a documentation tool provided by Java that allows developers to write documentation directly in the source code using specially formatted comments.

These comments are then processed by the Javadoc tool to generate HTML documentation for the code.

Syntax:

Javadoc comments begin with `/**` and end with `*/`.

Inside these comments, special tags are used to provide additional structured information.

Placement:

Javadoc comments are typically placed directly above the class, method, or field they document.

Common Tags:

`@param:` Describes a method parameter.

`@return:` Describes the return value of a method

`@deprecated:` Marks a method or class as outdated and suggests an alternative.

`@author:` Lists the author of the code.

`@since:` Indicates the version when the feature was introduced.

`@version:` Specifies the version of the class or method.

```

/**
 * Represents a book with a title
 *
 * @author Sarah Foss
 * @version 1.0
 * @since 2024-11-21
 */
public class Book {

    /**
     * The title of the book.
     */
    private String title;

    /**
     * Constructs a new Book with the specified title
     *
     * @param title the title of the book
     */
    public Book(String title) {
        this.title = title;
    }
}

```

```

/**
 * Gets the title of the book.
 *
 * @return the title of the book
 */
public String getTitle() {
    return title;
}

/**
 * Sets a new title for the book.
 *
 * @param title the new title of the book
 */
public void setTitle(String title) {
    this.title = title;
}

/**
 * Returns a string representation of the state.
 *
 * @return a string indicating the title
 */
public String toString() {
    return "Book Title: " + this.title;
}
}

```

Class Abstraction

Class abstraction is a programming concept that involves exposing only the essential features of a class while hiding its underlying implementation details.

It allows clients of the class to interact with it through a well-defined interface, focusing on what the class does rather than how it does it.

This is achieved by separating the external behavior of a class from its internal workings using:

Encapsulation: The internal data fields and behaviour that are not relevant to the client are hidden. This is done by marking them as private

Public Interface: The class provides public methods (and sometimes properties) that define how other parts of the program can interact with it.

Black Box: Users see the class as a "black box," relying on its public methods to perform tasks without needing to know the intricate details of how those tasks are accomplished internally.

Wrapper Classes

Wrapper classes provide a way to use primitive data types as objects.

These classes "wrap" primitive values in an object and provide several useful utility methods for converting between types, parsing, comparing, and more.

They can be also used in situations where only objects are allowed.

Each primitive type has a corresponding wrapper class.

Wrapper Classes

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

The Integer and Double Classes

java.lang.Integer
<code>-value: int</code> <code>+MAX VALUE: <u>int</u></code> <code>+MIN VALUE: <u>int</u></code>
<code>+Integer(value: int)</code> <code>+Integer(s: String)</code> <code>+byteValue(): byte</code> <code>+shortValue(): short</code> <code>+intValue(): int</code> <code>+longVlaue(): long</code> <code>+floatValue(): float</code> <code>+doubleValue():double</code> <code>+compareTo(o: Integer): int</code> <code>+toString(): String</code> <code>+<u>valueOf(s: String): Integer</u></code> <code>+<u>valueOf(s: String, radix: int): Integer</u></code> <code>+<u>parseInt(s: String): int</u></code> <code>+<u>parseInt(s: String, radix: int): int</u></code>

java.lang.Double
<code>-value: double</code> <code>+MAX VALUE: <u>double</u></code> <code>+MIN VALUE: <u>double</u></code>
<code>+Double(value: double)</code> <code>+Double(s: String)</code> <code>+byteValue(): byte</code> <code>+shortValue(): short</code> <code>+intValue(): int</code> <code>+longVlaue(): long</code> <code>+floatValue(): float</code> <code>+doubleValue():double</code> <code>+compareTo(o: Double): int</code> <code>+toString(): String</code> <code>+<u>valueOf(s: String): Double</u></code> <code>+<u>valueOf(s: String, radix: int): Double</u></code> <code>+<u>parseDouble(s: String): double</u></code> <code>+<u>parseDouble(s: String, radix: int): double</u></code>

Wrapper Classes

```
Integer i1 = Integer.valueOf(100);  
Integer i2 = Integer.valueOf("100");  
System.out.println(i1.compareTo(i2));  
System.out.println(i1.doubleValue());
```

```
Double d1 = Double.valueOf(3.14);  
Double d2 = Double.valueOf("47.3");  
System.out.println(d1.compareTo(d2));  
System.out.println(d1.intValue());
```

Autoboxing and Unboxing

Java supports autoboxing and unboxing, which automatically convert between primitive types and their corresponding wrapper classes:

- **Autoboxing:** The automatic conversion of a primitive type to its corresponding wrapper class.
- **Unboxing:** The automatic conversion of a wrapper class to its corresponding primitive type.

```
// Autoboxing: Converting int to Integer automatically  
Integer num = 10;
```

```
// Unboxing: Converting Integer to int automatically  
int val = num;
```

BigInteger

BigInteger is a class in the `java.math` package that provides operations for arbitrarily large integers.

Unlike primitive data types like `int` or `long`, which have fixed size limits, BigInteger can represent numbers of virtually unlimited size (limited only by available memory).

```
(import java.math.BigInteger;)
```

```
BigInteger big1 = new BigInteger("12345678901234567890");  
BigInteger big2 = new BigInteger("98765432109876543210");  
BigInteger sum = big1.add(big2);  
BigInteger product = big1.multiply(big2);  
System.out.println("Sum: " + sum);  
System.out.println("Product: " + product);
```

BigDecimal

BigDecimal is a Java class that provides operations for high-precision decimal numbers. It is ideal for financial and scientific calculations where accuracy is crucial.

```
(import java.math.BigInteger;)

BigDecimal bd1 = new BigDecimal("123.456789");
BigDecimal bd2 = new BigDecimal("987.654321");
BigDecimal difference = bd1.subtract(bd2);
BigDecimal product = bd1.multiply(bd2);
System.out.println("Difference: " + difference);
System.out.println("Product: " + product);
```

Strings

The String class in Java is one of the most widely used classes and is part of the `java.lang` package.

It represents a sequence of characters and provides a wide range of methods for string manipulation and analysis.

The String class has 13 constructors and more than 40 methods for manipulating strings.

Constructing Strings

While most strings are created using literals, there are multiple ways to construct strings:

From a String literal:

```
String str = "Hello";
```

Using the `new` keyword:

```
String str = new String("Hello");
```

From a char array:

```
char[] chars = {'H', 'e', 'l', 'l', 'o'};  
String str = new String(chars);
```

Strings are Immutable

Strings are immutable, which means once a String object is created, its content cannot be changed.

Immutable strings prevent accidental or malicious modifications, which is useful in areas like networking (e.g., URLs) or file paths.

Does the following code change the contents of the string?

```
String s = "I love ";  
s += "Java";
```

The String Pool

The **String Pool**, also known as the **String Intern Pool**, is a special memory region in the heap that stores string literals.

It is part of the JVM's optimization strategy for handling strings efficiently.

Since strings are immutable, if two or more strings have the same value, they can share the same memory in the pool rather than creating duplicate objects.

When a string is created as a literal (e.g., `String string1 = "Hello";`) , the JVM checks the String Pool:

- If the string already exists, its reference is returned.
- If the string does not exist, it is added to the pool.

Strings created using the `new` keyword are not automatically added to the pool unless explicitly **interned**.

The String Pool

```
String str1 = "Java";  
String str2 = "Java";  
String str3 = new String("Java");  
String str4 = str3.intern();
```

```
System.out.println(str1 == str2); // true (same reference)  
System.out.println(str1 == str3); // false (different reference)  
System.out.println(str1 == str4); // true (both refer to the pool object)
```

Splitting Strings

Splitting a string is commonly done using the `split()` method of the `String` class.

This method breaks a string into an array of substrings based on a specified **delimiter**.

-delimiter: string used to separate or divide text into smaller parts

Example:

```
String str = "apple,banana,cherry";  
String[] fruits = str.split(",");  
for (String fruit : fruits) {  
    System.out.println(fruit);  
}
```

StringBuilder

The StringBuilder class is an alternative to the String class.

Unlike a String, which is immutable and cannot be changed once created, StringBuilder is mutable, meaning it allows you to modify the state directly.

This includes adding, inserting, or appending new data into the existing sequence.

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: <i>aPrimitiveType</i>): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: <i>aPrimitiveType</i>): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

```
StringBuilder spell = new StringBuilder("Abra");

// Append additional text
spell.append(" cadabra!");
System.out.println("After append: " + spell); // Output: Abra cadabra!

// Insert magical text at a specific position
spell.insert(5, " alakazam");
System.out.println("After insert: " + spell); // Output: Abra alakazam cadabra!

// Replace a part of the text
spell.replace(5, 14, "hocus pocus");
System.out.println("After replace: " + spell); // Output: Abra hocus pocus cadabra!

// Delete a redundant portion of the text
spell.delete(17, 25); // Removes "cadabra!"
System.out.println("After delete: " + spell); // Output: Abra hocus pocus!

// Reverse the text
spell.reverse();
System.out.println("After reverse: " + spell); // Output: !sucop sucoh arbA
```

Summary

- A class is a blueprint for objects, defining properties, constructors, and methods for manipulating them. It acts as a data type, allowing the declaration of object reference variables.
- An object is an instance of a class, created using the new operator. Instance variables and methods belong to individual objects, while static variables and methods are shared across all instances of a class.
- Visibility modifiers control access to class members. Public members are accessible to all clients, while private members are restricted to the class. Getter (accessor) and setter (mutator) methods allow controlled access to private data.
- Java uses pass-by-value for parameters: primitives pass actual values, while reference types pass object references.
- Java arrays can hold primitive or object types, and they are initialized with default values upon creation.

Summary

- Immutable objects cannot be modified once created. Immutable classes ensure data integrity by preventing users from altering instances.
- Instance and static variables are scoped to the entire class and typically declared at the beginning of the class.
- The keyword `this` refers to the calling object and can invoke other constructors in the same class.
- Java's wrapper classes (e.g., `Integer`, `Double`) enable primitive data to be treated as objects. Auto-boxing and unboxing allow automatic conversion between primitives and wrappers.
- The `BigInteger` and `BigDecimal` classes handle computations involving very large integers or floating-point numbers with arbitrary precision.
- The `String` class is immutable. To optimize memory, Java stores identical string literals as unique, interned objects.
- The `StringBuilder` class offers mutable alternatives to `String`, used when string contents need modification.